Instituto Superior de Engenharia
Politécnico de Coimbra

Software Testing and Quality

- M.Sc. in Informatics Engineering

Overview

- **Introduction to Software Testing**
  - What is software testing? Why should we test? Who should do testing? How much should we test? …
- **Software Verification and Validation**
  - What is verification and validation? What are the differences? What are the limitations?…
- **Software Quality**
  - Role of process in software quality. Software control. Quality assurance. Quality factors…

Introduction to Software Testing

The Basics

What is Software Testing?

“Testing is the process of exercising or evaluating a system or system component by manual or automated means to verify that it satisfies specified requirements.”

IEEE

What is Software Testing?

- **We can start by trying to define testing as the process of executing a program with the intent of finding <u>faults</u>.**
  - *This is a very narrow definition, but it's good enough for now…*
- **Although it depends on the project, you typically may spend 40% of development time and 25% of budget on testing:**
  - *If you try to cut corners, you'll typicaly end up spending even more…*

What is Software Testing?

- **Testing is not debugging**
  - *Testing finds faults.*
  - *Debugging fixes faults.*
- **Program is supplied with valid and invalid inputs**
  - *We then compare actual results with <u>expected results</u>*
- **But this requires a running program…**
  - *What about requirements, and other non-executable artifacts?*

What is Software Testing?

- **Static Testing:** Analysis and inspection of development artifacts without program execution, typically by peer review:
  - ***Code Reviews :*** *Checking correctness of code by having a team of several people reading it.*
    - *PRs support code reviews.*
  - ***Walkthroughs****: Having the developer present and explain a software development artifact to a review team.*
  - ***Code Inspections and Sofware Technical Reviews****: Formalized processes for peer-reviewig code or other software artefacts.*

What is Software Testing?

- **Dynamic Testing:** Testing conducted by analysis of the output of running code.
  - Can start as soon as code is available to run
  - **Black-Box testing:** *When the internals of the system under test are not known or considered*
  - **White-Box testing:** *When the internals of the system under test are known and taken advantage of for testing purposes. Aka Glass-Box testing.*
  - **Gray-Box testing**: *An hybrid Black+White approach*

What is Software Testing?

Testing as Demonstration?:

- Showing correct operation with typical inputs.

- *“Testing hoping for success”*
- Testing *cannot* demonstrate that errors are not present.
- Testing *cannot* show that a program always performs its intended function correctly.
- How about “increasing confidence in correct operation”?
  - *Ok, but if we focus on this perspective only, we might emphasise creating tests that pass “successfully”, rather than tests that seek to identify problems.*

What is Software Testing?

1. **Positive Testing*:*** Testing the application under test (AUT) as it is intended to be operated.
   - *Legal inputs*
   - *Does the application behave normally?*
   - *Not “testing for sucess”. Still trying to make the AUT fail…*

2. **Negative Testing:** Testing the AUT by using it in ways that were not intended.
   - *Abnormal/malformed inputs.*
   - *Trying to deliberately push the system into doing things it shouldn’t do (or fail to do what it should so).*

What is Software Testing?

3. **Verification:** The process of identifying faults in requirements, design and the application under test (AUT) during earlier stages of development.
4. **Validation:** The process of identifying faults in the AUT or one of its modules or components at the end (or near the end) of its development..

Common Perspecive

Software Testing =
Software Verification + Software Validation

What is Software Testing?

Software verification:“*It is the process of evaluating, reviewing, inspecting and doing desk checks of work products such as requirement specifications, design specifications and code.*”
“*It is a human testing activity as it involves looking at the documents on paper.*”

IEEE

What is Software Testing?

Software validation:"It is defined as the process of evaluating a system or component during or at the end of development process to determine whether it satisfies the specified requirements. It involves executing the actual software. It is a computer based testing process."

IEEE

Why do We Test?

We test for a lot of different reasons:

Why do We Test?

The Technical Case:

- Competent developers are not infallible.
- The implications of requirements are not always foreseeable.
- The behavior of a system is not necessarily predictable from its components.
- Languages, databases, user interfaces, and operating systems have bugs that can cause application failures.
- Reusable classes and objects must be trustworthy.

Why do We Test?

The Business Case:

- If you don’t find bugs your customers or users will.
- Post-release debugging is the most expensive form of development.
- Buggy software hurts operations, sales, and reputation.
- Buggy software can be hazardous to life and property.

Why do We Test?

The Professional Case:

- Test case design is a challenging and rewarding task.
- Good testing allows confidence in your work.
- Systematic testing allows you to be most effective.
- Your credibility is increased and you have pride in your efforts.

Why do We Test?

The Safety Case:

- Bugs can cause huge losses.
- Bugs can cause disasters.
- Bugs can disable critical systems.
- Bugs can kill.

Why do We Test?

The Safety Case:

- Bugs can cause huge losses.
- Bugs can cause disasters.
- Bugs can disable critical systems.
- Bugs can kill.

Why do We Test?

The Quality Case:

- Testing provides can provide metrics for relevant quality factors.
- These factors may help improving not only the AUT, but even contribute to organizational or processual improvements.
- Some quality attributes (e.g. reliability) can be challenging to determined directly, but must instead be estimated via test results.

Why do We Test?

The Economic Case:

- Defects get introduced in every phase of Software development. They will propagate and be amplified by a certain factor if not corrected.
- The principle of *phase containement* of errors suggests identifying and removing errors in the originating phase, otherwise the cost of defect removal may increase.

Who Should Test?

Everyone has a role to play:

- Testing and development roles are important.
- Developers must contribute to testing efforts.
  - *Testing must not be an afterthough.*
  - *Artifacts must be created with testing in mind*
- Testing responsabilities should also be assigned to non-developers.
  - *Developers are usually biased towards creating "demonstration" tests, may carry misunderstandings to testing, may skip over "unnecessary" testing.*

How much should we test?

Consider the following code:

- How many different paths can be taken in a single iteration? And in 2 iterations? And in N iterations?
  - *3, 3x3= 3^2 , 3^N*

(a lot, not even considering the possibility of exceptions)

- **<u>Exhaustive testing is not viable.</u>**
  - Tests should focus on trouble spots or critical areas.

```
while(!done){
if(m[i]==0)
    {…}
else
    if(m[i]>0)
        {…}
    else
        {…}
i++;
}
```

When do we stop testing?

- **At some point, you need to stop testing and ship your product…**
  - The question is when?
  - When can you say you have done enough testing?
- **Because testing could potentially go on forever, this is basically a trade off between:**
  - Time
  - Budget
  - Quality

When do we stop testing?

- **Pessimistic Approach:**
  - Testing drains your time and budget. Test until you run out of some of these resources *(or until the unlikely event of running out of test cases)*

- **Optimistic Approach:**
  - Stop testing when reliability meets the requirements *(quality is high enough)*
  - Stop testing when the benefit does not justify the cost *(quality cannot be improved efficiently)*

Terminology

Error: A mistake, misconception, or misunderstanding on the part of a software developer. It is always human. An error can result in a …

Fault : An incorrect step, process or data definition in a computer program which causes the program to perform in an unintended or unanticipated manner. Faults lead to …

Failure : It is the inability of a system or component to perform the required function according to its specification.

Terminology

Incident: A symptom of a failure, or a potential symptom of a failure. This is the user-visible part of the failure, or just a result of normal operation that is surprising to the user. May warrant investigation. It may or may not actually reflect an underlying fault.

It's not a bug, it's a feature.

Terminology

Test Suite: Collection of test cases.
Test : The act of testing
Test Case (aka test): A test case is a description of expected program behavior. It has an identity, a purpose, a set of inputs and a set of expected outputs. Different test case templates are available and can be considered.

Test Case Template
Test ID
Purpose
Preconditions
Inputs
Expected Outputs
Postconditions
Execution History (Date, Result, Version, Run By)

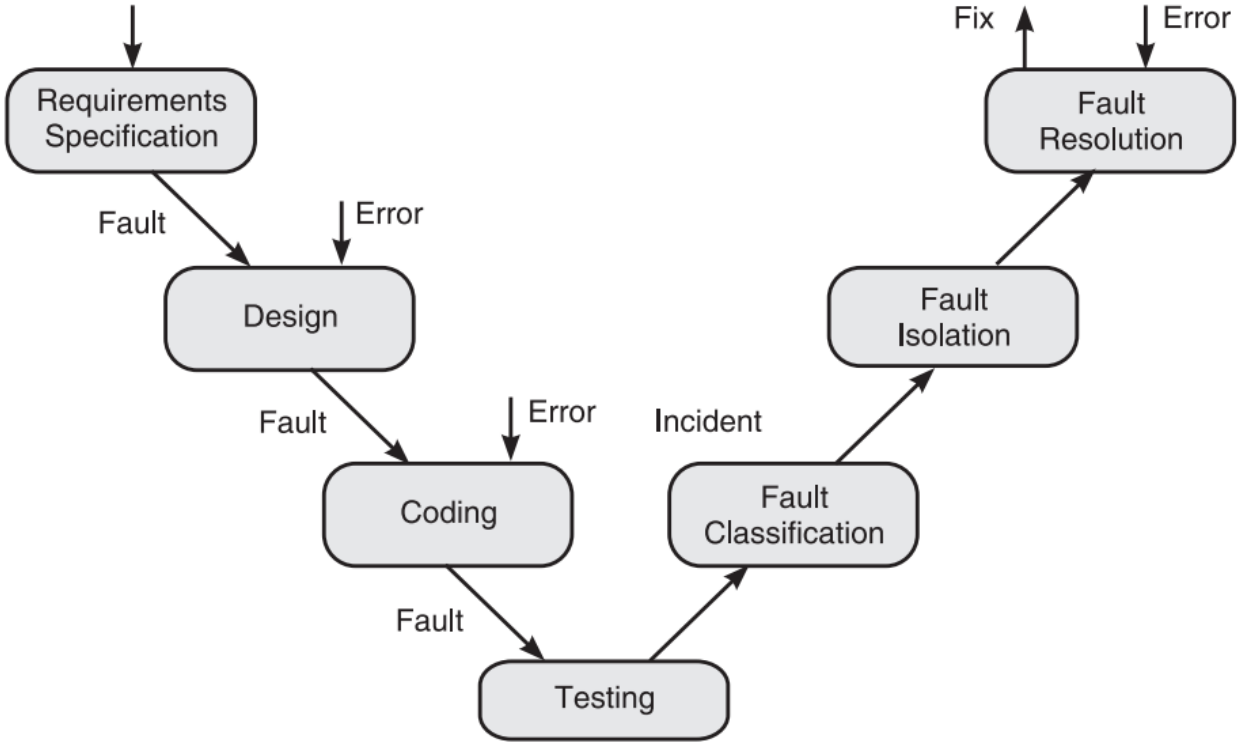
Terminology

Test Script: The instructions that actually execute one or more test cases

Test Suite: Collection of test cases (or test scripts).

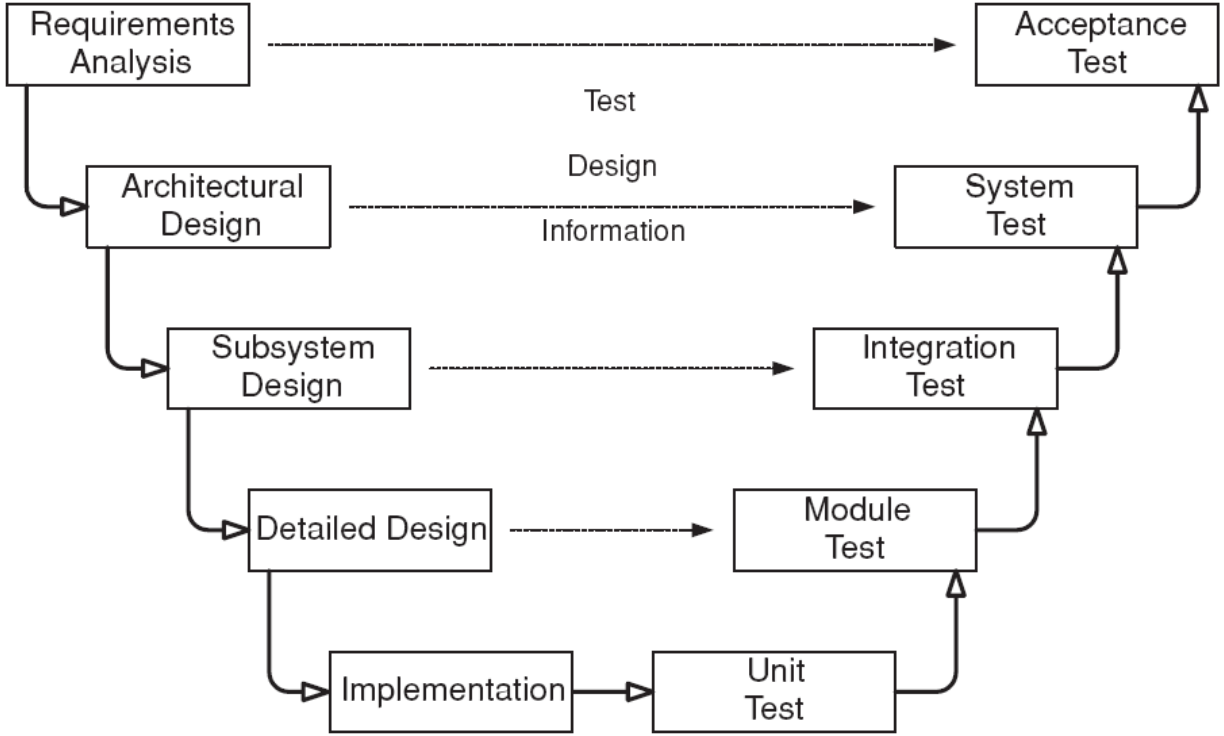
Unfortunately, the same terminology may be used differently in different sources. You have been warned…

A simple testing lifecycle



(from Software Quality Assurance, Chopra, 2018)

The V-Model

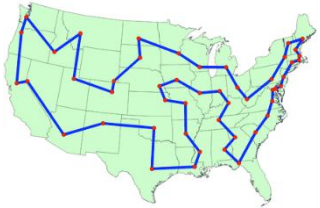

Requirements Analysis
Acceptance Test
Test
Design
Information
Architectural Design
System Test
Subsystem Design
Integration Test
Detailed Design
Module Test
Implementation
Unit Test

Limitations of Testing

- **Impracticality of testing all input data:** For most programs, it is impractical to attempt to test the program with all possible inputs due to a combinational explosion. Furthermore, a *testing oracle* (human or automated) must be available to determine the correctness of the output for a particular test input.

Input data

Traveling Salesman Optimizer



Output data

Is this the correct output?

Limitations of Testing

- **Impracticality of testing all paths:** For most programs, it is impractical to attempt to test all execution paths through the product due to a combinational explosion. It is also theoretically impossible to determine test data for achieving a specific path in arbitrary programs.

```
void f(double a, double b){
        Random r1=new Random((int)(a*b));
        Random r2=new Random((int)(1000+a-200*b*b));
        if(r1.nextInt()==r2.nextInt())
                {…}     // what values of a and b (if any)
                        // hit this path?
        else
                {…}
}
```

Limitations of Testing

- **No absolute proof of correctness:** It is not possible to prove a program is “correct”. It is only possible to prove that one description of something (i.e. the formal model of a system) is equivalent (in some way) to another description (i.e. the program code) – *proofs of equivalence*. The formal model would have to be assumed to be “correct” and that it would match user requirements. No general purpose testing or analysis procedure can be used to prove program correctness.

In summary…

- Testing not only aims to discover software faults, but also encompasses verification and validation and improving the reliability and quality of software.
- Testing time and resources are limited.
  - *Exhaustive testing is impossible.*
  - *Automation improves efficiency.*
  - *Aim for efficient testing: maximum coverage with minimum test-cases.*
- Testing is a team-effort.
  - *Developers (programmers) alone should not be responsible for all testing activities.*